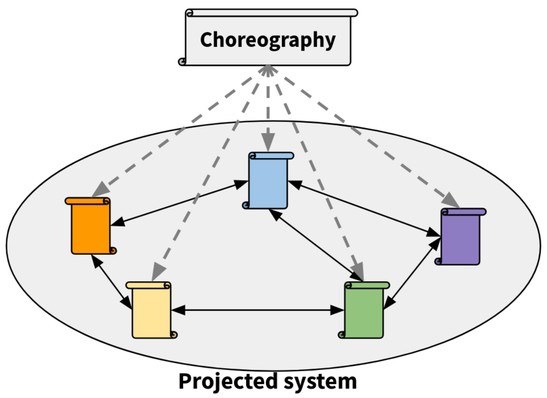
Choreographic Programming

Ashley Samuelson*, Andrey Yao*, Michael Piskozub†
Ethan Cecchetti*, Andrew K. Hirsch†

*University of Wisconsin - Madison †University at Buffalo

Introduction - Choreographies



Choreographic programming is developer friendly approach for handling distributed computing.

- Each node's computations and the interactions between nodes are described by a single top-level program: the choreography
- The choreography can be **projected** to a program for each individual location, so the programmer only needs to track one top-level program
- Many choreographic languages have **deadlock freedom by design**, so nodes will not be stuck waiting for messages that never arrive

Public-Key Cryptography

```
fun send_and_verify(msg : Bob.string) =
  let (public_key, secret_key) = Alice.[gen_key_pair()] in
  let Bob.pk = public_key ~> Bob in
  let Alice.enc = Bob.[encrypt(msg, pk)] ~> Alice in
  if Alice.[verify_msg(decrypt(enc, secret_key))]
    then Alice[Left] ~> Bob ; Bob.[print("success")]
    else Alice[Right] ~> Bob ; Bob.[print("failure")]

fun Alice.send_and_verify() =
  let (public_key, secret_key) = gen_key_pair() in
  send public_key to Bob ;
  let enc = receive from Bob in
  if verify_msg(decrypt(enc, secret_key))
    then choose Left for Bob
    else choose Right for Bob

fun Bob.send_and_verify(msg : string) =
  let pk = receive from Alice in
  send encrypt(msg, pk) to Alice ;
  allow Alice choice of
    Left  => print("success")
    Right => print("failure")
```

Ordinal-Indexed Operational Semantics

Choreographies give a global view of both intra- and inter-locational behaviors in the system. Although the projected system can execute out of order due to parallelism, if executed with a standard semantics the **choreography only captures a single execution order**. The existing techniques to mitigate this discrepancy include equivalence-based semantics, and limited out-of-order semantics for choreographies. However these semantics can be difficult to analyze, brittle, and are highly language-dependent.

We propose a choreographic semantics to solve this problem by using **traces** of events indexed by **ordinal numbers**. Ordinals are a number system which generalizes the natural numbers and allows us to consider events that happen "beyond infinity". The following example illustrates why choreographic programming is a natural setting for ordinals:

Visible v.s. Blocked Events

```
(fun F(X) = (Alice.[print("A")] ; F X)) () ;  // Alice loops
(fun G(Y) = (Bob.[print("B")] ; G Y)) () ;    // Bob loops
Alice.42 ~> Deidra ;                          // Event should be blocked
Charlie.[print("C")]                          // Event should be visible
```

Here, Alice prints an infinite stream of "A" and Bob does a similar thing in the following line. However, Alice should not be able to send any message to Deidra on line 3 because she is blocked by her infinite loop. Finally, Charlie would be able to print just fine. If the semantics were in-order, all of Bob's and Charlie's events would be lost because Alice never finishes executing, even though the two locations emitted completely independent events.

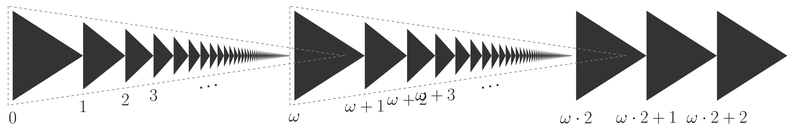
We formalized this concept as an operational semantics, where the steps are indexed by ordinal numbers. An example rule that allows for limit-ordinal-length traces is shown below.

$$t = \bigsqcup_{\lambda} t_\lambda \qquad \forall \lambda < \lambda_{\lim}.\, \langle C, \mathcal{B} \rangle \Longrightarrow_{t_\lambda}^{\lambda} \langle C_\lambda, \mathcal{B}_\lambda \rangle$$

$$\operatorname{limord}(\lambda_{\lim}) \qquad \mathcal{B} = \left(\bigcup_{\lambda} \mathcal{B}_\lambda \right) \cup \operatorname{infHosts}(t)$$

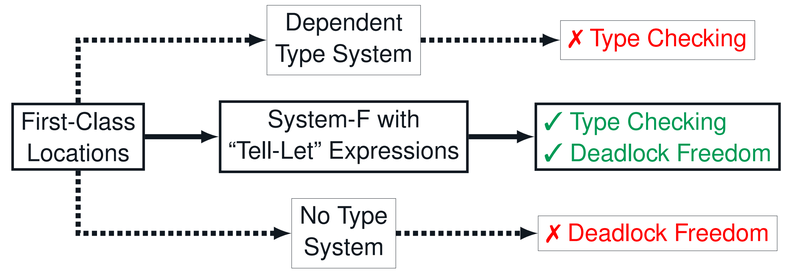
$$\frac{\forall E', C'.\, C = E'[C'] \wedge E' \neq [\cdot] \implies \exists \lambda < \lambda_{\lim}.\, \exists V, t_\lambda, \mathcal{B}''.\, \langle C', \mathcal{B} \rangle \Longrightarrow_{t_\lambda}^{\lambda} \langle V, \mathcal{B}'' \rangle}{\langle E[C], \mathcal{B} \rangle \Longrightarrow_{t}^{\lambda_{\lim}} \langle E[\bot], \mathcal{B}' \rangle}$$

Using these rules, the execution steps of the example program can be visualized using this diagram of ordinal numbers up to $\omega \cdot 2 + 2$. The execution trace is similarly a ordinal-length sequence of events.



First-Class Location Names

Many applications require us to be able to treat **location names** in a **first-class manner**. However, existing approaches to treating location names in a first-class manner all have significant drawbacks.



Our approach: **Reify first-class location names into type-level locations** using a combination of a let and send expression we call a "tell-let" expression. We are careful to avoid both deadlocks and type dependency using the following typing rule:

$$\frac{(1)\ \Gamma \vdash C_1 : \ell.\mathsf{loc}_\rho \quad (2)\ \rho \subseteq \rho' \quad (3)\ \Gamma, X :: \mathsf{Loc} \vdash C_2 : \tau \quad (4)\ \Gamma \vdash \tau :: \mathsf{Type}}{\Gamma \vdash \text{let } X = C_1 \rightsquigarrow \rho' \text{ in } C_2 : \tau}$$

(1) C_1 produces a first-class location name known by ℓ
(2) Anyone that the location name may resolve to must be informed by ℓ
(3) In C_2 the location name is bound to the type-level location X
(4) The type of C_2 may not depend on the resolved type-level location

Distributed Thread Pool

```
fun run_on_worker(task : Charlie.(s → t), data : Charlie.s) =
  let Manager.w = Manager.[acquire_worker()] in
  let W = Manager.w ~> {Charlie} ∪ pool in
  let W.f = task ~> W in
  let W.x = data ~> W in
  let result = W.[f(x)] ~> Charlie in
  W.["done"] ~> Manager ; Manager.[release_worker(w)] ; result

fun Charlie.run_on_worker(task : s → t, data : s) =
  let W = receive from Manager in
  send task to W ; send data to W ; receive from W

fun Worker.run_on_worker() =
  let W = receive from Manager in
  if I am W
    then let f = receive from Charlie in
         let x = receive from Charlie in
         send f(x) to Charlie ;
         send "done" to Manager
    else halt
```